

Week 6 Lecture Plan

Parallelism and Fork/Join: Granularity, Break-Even, and Pitfalls

Instructor: Dr. Mohamad Aoude

Pre-Class Preparation

Essential materials to prepare:

- ☐ Week 6 lab compiles: `week6-lab/week6-day1-parallelism-forkjoin`
- ☐ Smoke test passes: `gradlew.bat test -tests "edu.lu.concurrency.week6.day1.DemoSmokeTest"`
- ☐ Lecture PDFs ready: `Lecture_06_01_nar.pdf` through `Lecture_06_04_nar.pdf`
- ☐ Lab manual ready: `Lab_Week06_ParallelismForkJoin.pdf`
- ☐ Terminal ready for live demos: sequential sum, Fork/Join sum, common-pool context, break-even probe, granularity cost, parallel-stream anti-example
- ☐ Backup output for break-even runs, because timing varies by laptop and JVM state

Room setup:

- Projector with slides and terminal side-by-side
- One terminal in the Week 6 lab folder
- One editor window showing `part1_fork_join`, `part2_break_even`, and `part3_parallel_streams`
- Keep `EXERCISES.md`, `LAB_INSTRUCTIONS.md`, and `CONCURRENCY_SCORECARD.md` visible

Learning Targets

By the end of Week 6, students must be able to:

1. Distinguish bounded task execution from CPU-bound parallel decomposition.
2. Implement a `RecursiveTask` with a sequential base case and recursive split.
3. Explain how threshold choice controls task granularity and overhead.
4. Measure sequential vs Fork/Join execution without claiming universal speedup.
5. Diagnose why a parallel stream can be slower than a sequential primitive loop.
6. Explain why blocking I/O inside the common `ForkJoinPool` is a starvation risk.

Lecture Structure

Segment	Material	Goal
Part 1	<code>Lecture_06_01_nar</code>	Fork/Join model, <code>RecursiveTask</code> , work stealing, CPU-bound framing
Part 2	<code>Lecture_06_02_nar</code>	Granularity, thresholds, warm-up, break-even measurement
Part 3	<code>Lecture_06_03_nar</code>	Parallel-stream anti-example and common-pool costs
Part 4	<code>Lecture_06_04_nar</code>	Blocking hazards, optional advanced executor bridge, Week 7 handoff

Recommended Time Budget

Time	Activity	Notes
0–10 min	Week 5 handoff	Bounded pools control load; Week 6 asks whether CPU work should be split.
10–40 min	Part 1 slides and Demo01–Demo03	Sequential baseline, <code>RecursiveTask</code> , common pool.
40–75 min	Part 2 slides and Demo04–Demo05	Threshold experiments and measurement caveats.
75–100 min	Part 3 slides and Demo06	Parallel stream slowdown causes.
100–115 min	Part 4 slides and Demo07	Blocking hazard, advanced optional patterns, Week 7 bridge.
115–120 min	Exit ticket	Threshold, overhead, break-even, common pool, blocking risk.

Live Demo Checklist

- ☐ **Demo01:** Run the sequential baseline and make it the correctness reference.
- ☐ **Demo02:** Show the `RecursiveTask` base case, split, fork, compute, and join pattern.
- ☐ **Demo03:** Inspect common-pool parallelism and available processors.
- ☐ **Demo04:** Measure several sizes and ask where Fork/Join first wins, if anywhere.
- ☐ **Demo05:** Show how smaller thresholds create more leaf tasks.
- ☐ **Demo06:** Compare primitive sequential sum with boxed parallel stream.
- ☐ **Demo07:** State the blocking-in-common-pool starvation risk.

Student Work Contract

Students implement only:

- `Ex1_ForkJoinSum`
- `Ex2_BreakEvenReport`
- `Ex3_ParallelStreamDiagnosis`

They must run:

```
.\gradlew.bat studentCheck
.\gradlew.bat test --tests "edu.lu.concurrency.week6.day1.DemoSmokeTest"
```

They must also submit:

- completed `CONCURRENCY_SCORECARD.md`
- measurement table for sequential vs Fork/Join
- short break-even explanation with hardware/JDK caveats
- short diagnosis of the four parallel-stream slowdown causes

Instructor Notes

- Keep the sequential implementation visible. Parallel code has no value if it changes the result.
- Do not promise that Fork/Join will beat sequential on every machine. The learning goal is measurement and explanation.
- Emphasize that `threshold` is a design parameter, not a magic constant.
- Treat parallel streams as a convenience with costs, not as an automatic performance feature.
- Defer `CompletionService`, `FutureTask`, and memoization to optional discussion or Week 7 unless students are already comfortable with the main lab.
- Close with the Week 7 handoff: async design is about waiting, timeouts, fallbacks, and composition.

Exit Ticket

1. What is the sequential base case in a Fork/Join task?
2. Why can a very small threshold make Fork/Join slower?
3. What does a break-even point mean?
4. Name two costs that can make a parallel stream slower.
5. Why is blocking I/O inside the common `ForkJoinPool` risky?